

# AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

408/1, Kuratoli, Khilkhet, Dhaka 1229, Bangladesh



|                   |                              |                     |                                   |
|-------------------|------------------------------|---------------------|-----------------------------------|
| Assignment Title: | Mid Term Report              |                     |                                   |
| Assignment No:    | 01                           | Date of Submission: | November 30, 2025                 |
| Course Title:     | INTRODUCTION TO DATA SCIENCE |                     |                                   |
| Course Code:      |                              | Section:            | K                                 |
| Semester:         | Fall                         | 2025-26             | Course Teacher: Kamrun Naher Koli |

## Declaration and Statement of Authorship:

1. I/we hold a copy of this Assignment/Case-Study, which can be produced if the original is lost/damaged.
2. This Assignment/Case-Study is my/our original work and no part of it has been copied from any other student's work or from any other source except where due acknowledgement is made.
3. No part of this Assignment/Case-Study has been written for me/us by any other person except where such collaboration has been authorized by the concerned teacher and is clearly acknowledged in the assignment.
4. I/we have not previously submitted or currently submitting this work for any other course/unit.
5. This work may be reproduced, communicated, compared and archived for the purpose of detecting plagiarism.
6. I/we give permission for a copy of my/our marked work to be retained by the faculty for review and comparison, including review by external examiners.
7. I/we understand that Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to expulsion from the University. Plagiarized material can be drawn from, and presented in, written, graphic and visual form, including electronic data, and oral presentations. Plagiarism occurs when the origin of their material used is not appropriately cited.
8. I/we also understand that enabling plagiarism is the act of assisting or allowing another person to plagiarize or to copy my/our work.

\* Student(s) must complete all details except the faculty use part.

\*\* Please submit all assignments to your course teacher or the office of the concerned teacher.

Group Name/No.: I

| No | Name                        | ID         | Program | Signature |
|----|-----------------------------|------------|---------|-----------|
| 1  | Kayjer Islam                | 22-49005-3 | CSE     |           |
| 2  | Shanjida Sultana            | 23-51519-1 | CSE     |           |
| 3  | ANANNO, MD. REZOYANUL ISLAM | 23-50202-1 | CSE     |           |
| 4  |                             |            |         |           |
| 5  |                             |            |         |           |

## Faculty use only

|  |             |  |
|--|-------------|--|
|  |             |  |
|  |             |  |
|  |             |  |
|  | Total Marks |  |

## **Project Title:** Early Heart Attack Risk Prediction for Young Adults

**Data set details:** The dataset used in this project is a merged heart disease dataset consisting of 1,888 patient records, compiled from five publicly available and medically recognized heart disease datasets. It brings together a diverse range of clinical and demographic features commonly used in early cardiovascular risk prediction research. Each record represents an individual patient and includes 14 input features along with one binary target variable indicating heart attack risk.

The features cover key medical measurements such as resting blood pressure (*trestbps*), serum cholesterol levels (*chol*), maximum heart rate achieved (*thalach*), and ST depression values (*oldpeak*), as well as categorical diagnostic indicators like chest pain types (*cp*), electrocardiographic results (*restecg*), and thalassemia status (*thal*). Demographic attributes such as age and sex are also included. The target variable, *target*, classifies patients into two groups:

- 1 → Higher chance of heart attack
- 0 → Lower chance of heart attack

This combined dataset provides a comprehensive base for analyzing cardiovascular risk factors among adults. Although originally cleaned, intentional disruptions (missing values, invalid entries, duplicates, and outliers) were introduced as required by the project to simulate real-world data issues. These were later addressed through systematic data cleaning and preprocessing steps before exploratory analysis and predictive modeling.

### **Intentional Data Disruption Procedure:**

As instructed in the project guidelines, the dataset was intentionally disrupted to simulate real-world data quality issues commonly encountered in practical data science workflows. Although the original dataset was already clean and well-structured, introducing controlled imperfections allowed us to apply and demonstrate essential data preparation, cleaning, and preprocessing techniques. The disruptions were added carefully so that they remained realistic, medically plausible, and aligned with the types of errors that typically occur during data collection, manual entry, or dataset merging.

The following types of disruptions were intentionally introduced:

#### **1. Introduction of Missing Values**

A small, random portion (approximately 5%) of values was removed from both numerical attributes (such as *trestbps*, *chol*, *thalach*) and categorical variables (such as *sex*, *cp*, *restecg*). This simulates common issues such as incomplete patient records or missing sensor measurements.

#### **2. Insertion of Invalid or Impossible Values**

Several entries were deliberately replaced with values outside medically acceptable ranges, including:

- *age* values such as **-5** or **200**,
- *cholesterol* values such as **1500 mg/dl** or **-20 mg/dl**,
- *blood pressure* values such as **300 mmHg** or negative values.

These simulate human entry mistakes, device errors, or corrupted data in medical databases.

### 3. Creation of Duplicate Records

Randomly selected rows were duplicated and appended to the dataset to mimic scenarios where the same patient record appears multiple times due to merging errors or repeated submissions.

### 4. Formatting Inconsistencies in Categorical Variables

Some categorical fields were intentionally modified to include inconsistent text formatting, such as:

- "MALE", "male ", " Female", "X",
- "YES", "no ", "abc", "Unknown".

This reflects typical inconsistencies caused by manual typing variations or non-standard coding practices.

### 5. Addition of Extreme Outliers

Artificial outliers were inserted into continuous variables (e.g., *bmi*, *chol*, *oldpeak*) to simulate rare measurement errors or anomalous individuals in the sample.

### 6. Random Shuffling of Selected Values

A subset of values from selected columns (such as *age* or *thalach*) was shuffled between records to represent accidental misalignment or column mix-up during dataset merging.

#### R code for disrupting data set:

The CSV file was loaded into an R **data frame** using the `read.csv()` function.

```
df <- read_csv("cleaned_merged_heart_dataset.csv")
```

#### Code Explanation:

We used the `read.csv()` function to **import** the CSV data and create a structured **data frame** in R. By calling `read.csv()`, the CSV dataset was **structured** and **stored** as an R data frame object.

#### Dataset:

The dataset is loaded with all columns as **character type (text)** to prevent R from automatically converting invalid or inconsistent values into NA. Printing the column names ensures all variables are correctly recognized before introducing disruptions.

```

library(tidyverse)

df <- read_csv("cleaned_merged_heart_dataset.csv",
               col_types = cols(.default = "c"))

print(names(df))

> library(tidyverse)
>
> df <- read_csv("cleaned_merged_heart_dataset.csv",
+               col_types = cols(.default = "c"))
>
> print(names(df))
[1] "age"      "sex"      "cp"      "trestbps" "chol"     "fbs"     "restecg"
[8] "thalachh" "exang"    "oldpeak" "slope"    "ca"       "thal"    "target"
>

```

#### Code Explanation:

library(tidyverse) loads useful packages for data handling, including readr and dplyr.  
 read\_csv(..., col\_types = "c") loads every column as a character so that invalid values like “two” or “FEMALE ” do NOT get converted into NA.  
 print(names(df)) prints all column names so we confirm the dataset structure before making modifications.

#### Inserting missing values:

Randomly selected entries (5% of each column) were replaced with NA to simulate incomplete patient records, a common issue in real-world healthcare datasets.

```

set.seed(50)

# 1. Insert Missing Values
for (col in names(df)) {

  idx <- sample(nrow(df), size = floor(0.05 * nrow(df)))
  df[[col]][idx] <- NA
}

set.seed(50)

# 1. Insert Missing Values
for (col in names(df)) {

  idx <- sample(nrow(df), size = floor(0.05 * nrow(df)))
  df[[col]][idx] <- NA
}

```

**Code explanation:**

**set.seed(50)** makes the random selection reproducible (same missing values every time).

**for (col in names(df))** loops through each column.

**sample()** chooses 5% of rows randomly.

**df[[col]][idx] <- NA** removes those values, simulating real-world missing medical data such as incomplete patient files.

**Inserting invalid values:**

These artificial invalid medical values (e.g., negative age, extremely high cholesterol) imitate human errors, faulty devices, or data entry issues. Such invalid values must later be detected and corrected during data cleaning.

```
# 2. Insert Invalid Values
if ("age" %in% names(df)) {
  df$age[sample(nrow(df), 5)] <- c("-5", "150", "200", "0", "300")
}

if ("chol" %in% names(df)) {
  df$chol[sample(nrow(df), 3)] <- c("1500", "-20", "0")
}

if ("trestbps" %in% names(df)) {
  df$trestbps[sample(nrow(df), 3)] <- c("300", "-10", "999")
}

# 2. Insert Invalid Values
if ("age" %in% names(df)) {
  df$age[sample(nrow(df), 5)] <- c("-5", "150", "200", "0", "300")
}

if ("chol" %in% names(df)) {
  df$chol[sample(nrow(df), 3)] <- c("1500", "-20", "0")
}

if ("trestbps" %in% names(df)) {
  df$trestbps[sample(nrow(df), 3)] <- c("300", "-10", "999")
}
```

**Code explanation:**

These commands replace a few valid numbers with impossible or unrealistic values such as:

- Negative age
- Very high cholesterol (1500 mg/dl)
- BP = -10 mmHg or 999 mmHg

This mimics data entry mistakes, sensor errors, or merging issues found in real medical datasets.

Later, in the cleaning phase, these invalid observations will be detected and corrected or removed.

**Inserting outliers:**

Unusually high or low values were introduced to simulate outliers caused by equipment malfunction or incorrect recording. These values are far outside normal physiological ranges and will be identified during outlier detection.

```
# 3. Insert Outliers
```

```
if ("thalach" %in% names(df)) {
  df$thalach[sample(nrow(df), 3)] <- c("350", "400", "20")
}
```

```
# 3. Insert Outliers
```

```
if ("thalach" %in% names(df)) {
  df$thalach[sample(nrow(df), 3)] <- c("350", "400", "20")
}
```

**Code Explanation:**

thalach = maximum heart rate. Normal values range from **60–200**. Values like **350, 400, and 20** are extreme outliers. These represent faulty medical equipment or wrong measurement units. Your cleaning section will later treat these as outliers.

**Inconsistent categorical values:**

These inconsistent text formats replicate common issues in categorical data such as extra spaces, different capitalization, or invalid category labels. These must later be standardized or corrected.

```
# 4. Inconsistent Categorical Values
```

```
if ("sex" %in% names(df)) {
  df$sex[sample(nrow(df), 5)] <- c("MALE", " male", "FEMALE ", "X", "unknown")
}
```

```
if ("cp" %in% names(df)) {
  df$cp[sample(nrow(df), 5)] <- c("0 ", "1", "two", "Three", "unknown")
}
```

```
# 4. Inconsistent Categorical Values
```

```
if ("sex" %in% names(df)) {
  df$sex[sample(nrow(df), 5)] <- c("MALE", " male", "FEMALE ", "X", "unknown")
}
```

```
if ("cp" %in% names(df)) {
  df$cp[sample(nrow(df), 5)] <- c("0 ", "1", "two", "Three", "unknown")
}
```

**Code explanation:**

Introduces errors common in human typing: Extra spaces: " male", "FEMALE " Wrong capitalization:

"MALE" Wrong categories: "X", "unknown" Non-numeric labels for numeric category ("two", "Three")  
These inconsistencies allow you to demonstrate recoding trimming spaces standardizing categories.

#### **Duplication:**

Duplicates simulate errors often caused by merging multiple datasets or repeated patient entries. These duplicates will be detected and removed during data cleaning.

```
# 5. Add duplicates
```

```
dup <- df[sample(nrow(df), 20), ]  
df <- bind_rows(df, dup)
```

```
# 5. Add duplicates
```

```
dup <- df[sample(nrow(df), 20), ]  
df <- bind_rows(df, dup)
```

#### **Code explanation:**

`sample()` selects 20 random rows. `bind_rows()` appends them to the dataset.  
This creates duplicate patient records, simulating: merging errors, repeated entries ,system synchronization issues

#### **Saving the corrupted dataset:**

The final artificially disrupted dataset is saved for the next stage of the project: data cleaning and preprocessing.

```
write_csv(df, "corrupted_heart_dataset.csv")  
  
cat("Corrupted dataset created successfully!")  
> write_csv(df, "corrupted_heart_dataset.csv")  
>  
> cat("Corrupted dataset created successfully!")  
Corrupted dataset created successfully!  
.
```

#### **Code explanation:**

Saves the final intentionally disrupted dataset. This file is now used for the data cleaning section of the project. The printed message confirms the process was completed successfully.

## Dataset Cleaning:

### Loading the Dataset

The dataset was imported into R using the `read.csv()` function, which reads CSV files into a structured data frame. The `na.strings = "NA"` argument ensures that missing values are properly recognized for further processing.

```
> corrupted_heart_dataset <- read.csv(  
  "E:/Academics/8th Semester/Data Science/corrupted_heart_dataset.csv",  
  header = TRUE,  
  na.strings = "NA"  
)
```

### Code Explanation:

The `read.csv()` function is used to load the heart disease dataset from the specified file path into R as a data frame named `corrupted_heart_dataset`. The argument `header = TRUE` tells R that the first row of the file contains column names rather than data values. The `na.strings = "NA"` option converts any "NA" text in the file into proper missing values (NA) in R, ensuring that incomplete entries can be correctly detected and handled later.

### Dataset Explanation:

Basic inspection functions were used to understand the dataset's size, structure, and content.

`dim()` returned the number of rows and columns, `names()` listed variable names, `head()` showed initial records, and `summary()` provided quick statistical information.

### Code:

```
>dim(corrupted_heart_dataset)  
  
>names(corrupted_heart_dataset)  
  
> head(corrupted_heart_dataset)  
  
> summary(corrupted_heart_dataset)
```



```

> dim(corrupted_heart_dataset)
[1] 1908 14
> names(corrupted_heart_dataset)
[1] "age" "sex" "cp" "trestbps" "chol" "fbs" "restecg" "thalachh" "exang" "oldpeak" "slope" "ca"
[13] "thal" "target"
> head(corrupted_heart_dataset)
  age sex cp trestbps chol fbs restecg thalachh exang oldpeak slope ca thal target
1  63  1  3    150    233  1      0     150      0   2.3    0  0  1  1
2  37  1 <NA>    130    250  0      1     187      0   NA    0  0  2  1
3  41  0  1    130    204  0      0     172      0   1.4    2  0  2  1
4  56  1 <NA>    120    236  0      1     178      0   0.8    2  0  2  1
5 200  0  0    120    354  0      1     163      1   0.6    2  0  2  1
6  57  1  0    140    192  0      1     148      0   0.4    1  0  1  1
> summary(corrupted_heart_dataset)
      age      sex      cp      trestbps      chol      fbs      restecg      thalachh
Min.   : -5.00 Length:1908 Length:1908 Min.   : -10  Min.   : -20.0 Min.   : 0.0000 Min.   : 0.000 Min.   : 71.0
1st Qu.: 48.00 Class :character Class:character 1st Qu.:120 1st Qu.: 211.0 1st Qu.: 0.0000 1st Qu.: 0.000 1st Qu.:132.0
Median : 56.00 Mode  :character Mode  :character Median :130 Median : 240.0 Median : 0.0000 Median : 1.000 Median :152.0
Mean   : 54.63                                     Mean :132 Mean   : 247.1 Mean : 0.1529 Mean : 0.594 Mean :149.3
3rd Qu.: 61.00                                     3rd Qu.:140 3rd Qu.: 276.0 3rd Qu.: 0.0000 3rd Qu.: 1.000 3rd Qu.:166.0
Max.   :300.00                                     Max.   :999 Max.   :1500.0 Max. : 1.0000 Max. : 2.000 Max. :202.0
NA's   :95                                         NA's   :95 NA's   :95 NA's   :96 NA's   :95 NA's   :97

      exang      oldpeak      slope      ca      thal      target
Min.   : 0.0000 Min.   : 0.000 Min.   : 0.000 Min.   : 0.0000 Min.   : 0.000 Min.   : 0.0000
1st Qu.: 0.0000 1st Qu.: 0.000 1st Qu.: 1.000 1st Qu.: 0.0000 1st Qu.: 2.000 1st Qu.: 0.0000
Median : 0.0000 Median : 0.800 Median : 1.000 Median : 0.0000 Median : 2.000 Median : 1.0000
Mean   : 0.3342 Mean   : 1.065 Mean : 1.426 Mean : 0.7326 Mean : 2.672 Mean : 0.5182
3rd Qu.: 1.0000 3rd Qu.: 1.800 3rd Qu.: 2.000 3rd Qu.: 1.0000 3rd Qu.: 3.000 3rd Qu.: 1.0000
Max.   : 1.0000 Max.   : 6.200 Max.   : 3.000 Max.   : 4.0000 Max.   : 7.000 Max.   : 1.0000
NA's   :95      NA's   :94      NA's   :97 NA's   :94 NA's   :96 NA's   :94

```

## Code Explanation:

`dim(heart)` displays the total number of rows and columns in the dataset, giving an overview of its size. `names(heart)` lists all the variable names present in the dataset. `head(heart)` shows the first six rows, allowing a quick look at how the data is structured. `summary(heart)` provides basic statistical summaries such as minimum, maximum, median, and quartiles for each variable, helping identify unusual values and missing data.

## Handling Duplicate Values:

Duplicate records were removed to avoid repeated observations affecting the analysis. The initial row count was compared with the row count after applying the `unique()` function, allowing the number of removed duplicates to be calculated.

### Code:

```

heart <- corrupted_heart_dataset

> original_rows <- nrow(heart)

> heart <- unique(heart)

> new_rows <- nrow(heart)

> cat("Duplicate rows removed:", original_rows - new_rows)

```

```
> heart <- corrupted_heart_dataset
>
> original_rows <- nrow(heart)
> heart <- unique(heart)
>
> new_rows <- nrow(heart)
> cat("Duplicate rows removed:", original_rows - new_rows)
Duplicate rows removed: 546>
```

### Code Explanation:

The `nrow()` function is used to record the number of rows before and after duplicate removal. The `unique()` function removes all duplicate entries from the dataset. By comparing the original and new row counts, the number of removed duplicate rows is determined, ensuring the dataset contains only unique records.

### Finding Missing Values:

The presence of missing values was assessed using `colSums(is.na())`, which counts the number of NA entries in each column. This step helped determine which variables required imputation.

#### Code:

```
> colSums(is.na(heart))
```

```
> colSums(is.na(heart))
  age    sex    cp trestbps   chol    fbs  restecg thalachh   exang  oldpeak  slope    ca   thal  target
  92    92    92    93      92    91    93      93      94     92     92    92    91     91
```

### Code Explanation:

`is.na(heart)` identifies all missing values in the dataset by marking each NA entry as TRUE. `colSums()` then counts how many TRUE values appear in each column, producing a clear summary of how many missing values each variable contains.

### Handling Missing Value:

Missing values were filled using appropriate imputation methods.

Numerical columns were imputed using the median, while categorical variables were filled with the mode. A custom function was used to compute the mode. This ensured that all missing entries were replaced with representative values. After imputation, the dataset was checked again to confirm that no missing values remained. The output displayed zeros for all columns, indicating that the dataset was now complete and ready for further processing.

Code:

```
get_mode <- function(x) {  
  names(which.max(table(x)))  
}  
  
heart$age[is.na(heart$age)] <- median(heart$age, na.rm = TRUE)  
heart$restbps[is.na(heart$restbps)] <- median(heart$restbps, na.rm = TRUE)  
heart$chol[is.na(heart$chol)] <- median(heart$chol, na.rm = TRUE)  
heart$thalachh[is.na(heart$thalachh)] <- median(heart$thalachh, na.rm = TRUE)  
heart$oldpeak[is.na(heart$oldpeak)] <- median(heart$oldpeak, na.rm = TRUE)  
  
heart$sex[is.na(heart$sex)] <- get_mode(heart$sex)  
heart$cp[is.na(heart$cp)] <- get_mode(heart$cp)  
heart$fbs[is.na(heart$fbs)] <- get_mode(heart$fbs)  
heart$restecg[is.na(heart$restecg)] <- get_mode(heart$restecg)  
heart$exang[is.na(heart$exang)] <- get_mode(heart$exang)  
heart$slope[is.na(heart$slope)] <- get_mode(heart$slope)  
heart$ca[is.na(heart$ca)] <- get_mode(heart$ca)  
heart$thal[is.na(heart$thal)] <- get_mode(heart$thal)  
heart$target[is.na(heart$target)] <- get_mode(heart$target)  
  
colSums(is.na(heart))
```

```

> get_mode <- function(x) {
+   names(which.max(table(x)))
+ }
> heart$age[is.na(heart$age)] <- median(heart$age, na.rm = TRUE)
> heart$trestbps[is.na(heart$trestbps)] <- median(heart$trestbps, na.rm = TRUE)
> heart$chol[is.na(heart$chol)] <- median(heart$chol, na.rm = TRUE)
> heart$thalachh[is.na(heart$thalachh)] <- median(heart$thalachh, na.rm = TRUE)
> heart$oldpeak[is.na(heart$oldpeak)] <- median(heart$oldpeak, na.rm = TRUE)
> heart$sex[is.na(heart$sex)] <- get_mode(heart$sex)
> heart$cp[is.na(heart$cp)] <- get_mode(heart$cp)
> heart$fbs[is.na(heart$fbs)] <- get_mode(heart$fbs)
> heart$restecg[is.na(heart$restecg)] <- get_mode(heart$restecg)
> heart$exang[is.na(heart$exang)] <- get_mode(heart$exang)
> heart$slope[is.na(heart$slope)] <- get_mode(heart$slope)
> heart$ca[is.na(heart$ca)] <- get_mode(heart$ca)
> heart$thal[is.na(heart$thal)] <- get_mode(heart$thal)
> heart$target[is.na(heart$target)] <- get_mode(heart$target)
> colSums(is.na(heart))
      age      sex      cp trestbps      chol      fbs      restecg      thalachh      exang      oldpeak      slope      ca      thal
target
      0
> |

```

### Code Explanation:

A custom function `get_mode()` is created to calculate the mode, which is the most frequently occurring value in a column. Numerical columns such as age and cholesterol are filled with their median values, using `median(heart$..., na.rm = TRUE)` to ignore missing values during calculation. Categorical variables are imputed with their mode using the custom function. By replacing all NA entries with appropriate statistical values, the dataset becomes complete and free of missing data. The `colSums(is.na(heart))` command is used again after imputation to confirm that all missing values have been successfully replaced. A result of zero for every column indicates that the dataset now contains no NA values.

### Analysis of the Dataset:

The analysis begins by loading and cleaning a heart disease dataset, converting key medical variables such as age, cholesterol, blood pressure, and maximum heart rate into numeric form and transforming the target variable into a categorical factor. First, descriptive statistics (mean, median, and standard deviation) are calculated separately for the two target classes to compare how age, cholesterol level, and resting blood pressure differ between low-risk and high-risk heart patients. Next, the analysis compares average cholesterol levels between males and females to observe sex-based differences in cholesterol risk. Finally, it examines how maximum heart rate varies across different chest pain types by computing the mean, standard deviation, minimum, and maximum values, helping identify how heart performance changes across patient groups.

**Load Dataset:** We begin by loading R with necessary packages and the dataset.

```

library(tidyverse)

heart <- read.csv("cleaned_merged_heart_dataset.csv")

num_cols <- c("age", "trestbps", "chol", "thalach", "oldpeak")

num_cols <- intersect(num_cols, names(heart))

heart[num_cols] <- lapply(heart[num_cols], as.numeric)

if ("target" %in% names(heart)) {
  heart$target <- as.factor(heart$target)
}

```

```

View(cleaned_merged_heart_dataset)
library(tidyverse)

heart <- read.csv("cleaned_merged_heart_dataset.csv")

num_cols <- c("age", "trestbps", "chol", "thalach", "oldpeak")

num_cols <- intersect(num_cols, names(heart))

heart[num_cols] <- lapply(heart[num_cols], as.numeric)

if ("target" %in% names(heart)) {
  heart$target <- as.factor(heart$target)
}

```

Code Explanation: This code loads the **tidyverse** package and reads the heart disease dataset into a data frame called **heart**. It then defines a list of numeric column names (**num\_cols**) and uses **intersect()** to keep only the column names that actually exist in the dataset, preventing errors. Next, it converts all these selected columns to numeric using **lapply()**. Finally, the code checks whether the column **"target"** exists in the dataset, and if it does, it converts that column into a factor, which is useful for classification tasks. Overall, the code safely cleans and prepares the dataset by ensuring numerical columns are correctly typed and the target variable is properly categorized.

**Descriptive statistics across two target classes:** We compare age, chol (cholesterol), trestbps (blood pressure) Between: target = 0 → Low risk target = 1 → High risk  
 If mean\_age of target=1 > target=0 → High-risk patients are older on average. If mean\_chol is higher in target=1 → High cholesterol contributes to heart attack risk.

```

library(dplyr)

descriptive_stats <- heart %>%
  group_by(target) %>%
  summarise(
    mean_age = mean(age, na.rm=TRUE),
    median_age = median(age, na.rm=TRUE),
    sd_age = sd(age, na.rm=TRUE),

    mean_chol = mean(chol, na.rm=TRUE),
    median_chol = median(chol, na.rm=TRUE),
    sd_chol = sd(chol, na.rm=TRUE),

    mean_trestbps = mean(trestbps, na.rm=TRUE),
    median_trestbps = median(trestbps, na.rm=TRUE),
    sd_trestbps = sd(trestbps, na.rm=TRUE)
  )

descriptive_stats

```

```

library(dplyr)

descriptive_stats <- heart %>%
  group_by(target) %>%
  summarise(
    mean_age = mean(age, na.rm=TRUE),
    median_age = median(age, na.rm=TRUE),
    sd_age = sd(age, na.rm=TRUE),

    mean_chol = mean(chol, na.rm=TRUE),
    median_chol = median(chol, na.rm=TRUE),
    sd_chol = sd(chol, na.rm=TRUE),

    mean_trestbps = mean(trestbps, na.rm=TRUE),
    median_trestbps = median(trestbps, na.rm=TRUE),
    sd_trestbps = sd(trestbps, na.rm=TRUE)
  )

descriptive_stats
# A tibble: 2 x 10
#   target mean_age median_age sd_age mean_chol median_chol sd_chol mean_trestbps
#   <dbl>   <dbl>     <dbl>  <dbl>   <dbl>     <dbl>  <dbl>   <dbl>
#1     0     55.9       57   8.33    251.      248    50.1    133.
#2     1     52.9       53   9.51    243.      235    52.7    130.
#> 2 more variables: median_trestbps <dbl>, sd_trestbps <dbl>

```

**Code Explanation:** The **dplyr** package is used to summarize key numerical variables in the dataset by grouping them according to the **target** variable, which represents heart disease status. The **%>%** operator (pipe) passes the dataset step-by-step into each function, improving readability. First, **group\_by(target)** divides the dataset into two groups based on the value of **target** (0 = no heart disease, 1 = heart disease). Then, **summarise()** is used to compute summary statistics for each group. Inside **summarise()**, functions like **mean()**, **median()**, and **sd()** calculate the average, central value, and spread (standard deviation) of each variable such as age, cholesterol (chol), and resting blood pressure (trestbps). Each function uses **na.rm = TRUE** to remove missing values before calculation. The final output, stored in **descriptive\_stats**, is a table showing how these three numerical health indicators differ between the two target categories.

**Compare the average of one variable between two groups:** If men have higher average cholesterol → Men are at higher cholesterol-related risk. If women have lower values → Their lipid profile is comparatively healthier.

```
heart$sex <- as.factor(heart$sex)
```

```

avg_chol_by_sex <- heart %>%
  group_by(sex) %>%
  summarise(mean_chol = mean(chol, na.rm=TRUE))

```

```
avg_chol_by_sex
```

```
heart$sex <- as.factor(heart$sex)

avg_chol_by_sex <- heart %>%
  group_by(sex) %>%
  summarise(mean_chol = mean(chol, na.rm=TRUE))

avg_chol_by_sex
# A tibble: 2 × 2
#   sex    mean_chol
#   <fct>    <dbl>
#1 0         262.
#2 1         240.
```

**Code Explanation:** Converting the `sex` column into a factor using `as.factor()`, ensuring that R treats male and female as categorical groups rather than numeric values. The dataset is then passed into a sequence of functions using the pipe operator `%>%`. The `group_by(sex)` function divides the data into separate groups based on sex (typically 0 = female, 1 = male). After grouping, the `summarise()` function is used to compute the average cholesterol level for each sex category. Inside it, the `mean()` function calculates the mean cholesterol value, while `na.rm = TRUE` ensures that any missing cholesterol values are ignored during the computation. The resulting object, `avg_chol_by_sex`, is a summary table showing how average cholesterol differs between males and females in the dataset.

**Comparing variation of a numeric variable across categories:** Higher mean\_thalachh → better heart performance. Chest pain type (cp=0,1,2,3) often shows different HR patterns. If SD is large → HR varies a lot in that group meaning inconsistency in heart condition.

```
heart$cp <- as.factor(heart$cp)

variation_thalachh_by_cp <- heart %>%
  group_by(cp) %>%
  summarise(
    mean_thalachh = mean(thalachh, na.rm=TRUE),
    sd_thalachh = sd(thalachh, na.rm=TRUE),
    min_thalachh = min(thalachh, na.rm=TRUE),
    max_thalachh = max(thalachh, na.rm=TRUE)
  )

variation_thalachh_by_cp
# A tibble: 4 × 5
```

```

> heart$cp <- as.factor(heart$cp)
>
> variation_thalachh_by_cp <- heart %>%
+   group_by(cp) %>%
+   summarise(
+     mean_thalachh = mean(thalachh, na.rm=TRUE),
+     sd_thalachh = sd(thalachh, na.rm=TRUE),
+     min_thalachh = min(thalachh, na.rm=TRUE),
+     max_thalachh = max(thalachh, na.rm=TRUE)
+   )
>
> variation_thalachh_by_cp
# A tibble: 4 × 5
  cp    mean_thalachh sd_thalachh min_thalachh max_thalachh
<fct>      <dbl>      <dbl>      <int>      <int>
1 0          141.        23.1         71        190
2 1          162.        19.5        103        202
3 2          155.        19.2         96        194
4 3          149.        23.5         71        190
> |

```

**Code Explanation:** Converting the `cp` (chest pain type) column into a categorical variable using `as.factor()`, which ensures that R treats each chest pain type as a separate category. The dataset is then processed using the pipe operator `%>%`, which passes it step-by-step through the analysis. The `group_by(cp)` function splits the dataset into multiple groups based on chest pain type (e.g., `cp = 0, 1, 2, 3`). After grouping, the `summarise()` function calculates several measures of variation for the numerical variable `thalachh` (maximum heart rate achieved). Inside `summarise()`, the `mean()`, `sd()`, `min()`, and `max()` functions compute, respectively, the average, standard deviation, lowest value, and highest value of `thalachh` for each chest pain category. The argument `na.rm = TRUE` ensures missing values are ignored during calculations. The final output, `variation_thalachh_by_cp`, provides a clear comparison of how heart rate performance varies across different chest pain types.

**Dataset Ref:** <https://www.kaggle.com/datasets/mfarhaannazirkhan/heart-dataset>